

MANUAL DE TRABALHO

Entender.
Experimentar.
Construir juntos.

ALGORITMOS E ESTRUTURAS DE DADOS I

Um guia passo a passo para implementar os quatro algoritmos, medir seu comportamento e transformar os resultados em uma análise clara.

01 / Código	02 / Experimentos	03 / Colaboração
Correção e contratos	Tempo e operações	GitHub direto na main
Inversão e buscas	Dados, gráficos e teoria	Integração e entrega

Feito para todos os membros do projeto. Leia na ordem na primeira vez; depois use o sumário e os marcadores do PDF como referência.

Edição de apoio • 17 de setembro de 2026

O que este material representa

O guia combina o resumo do trabalho fornecido na conversa, o código local e a estrutura exibida na imagem. As melhorias de método são recomendações. O enunciado oficial completo não foi disponibilizado; ele prevalece em requisitos de entrada, saída, prazo e formato de entrega.

ROTEIRO DE LEITURA

Do primeiro teste à entrega

Páginas	O que você vai encontrar
3–5	Escopo, estado do projeto, pastas e responsabilidades
6–8	Correção, funcionamento dos algoritmos e desenho experimental
9–13	Contagem de operações e fórmulas verificáveis
14–17	Cronômetro, repetições, rodadas e interpretação estatística
18–20	CSV, gráficos e análise teórica
21–23	GitHub: primeiro acesso, rotina na main e conflitos
24–25	Relatório, integração e checklist de entrega
26–29	Exemplo completo de experimento em C, pronto para estudar
30	Referências e decisões a confirmar

Se você vai começar agora

1. Leia o contrato das funções e combine o padrão de contagem com a equipe.
2. Faça os testes pequenos antes de medir desempenho.
3. Estude um experimento completo e só depois adapte o seu algoritmo.
4. Salve os dados brutos, interprete os gráficos e escreva sua parte.
5. Atualize e envie alterações em pequenos commits na main.

Três palavras que não significam a mesma coisa

Teste: verifica se o resultado está certo. **Experimento:** mede comportamento sob condições definidas.

Análise: explica o resultado e o relaciona à teoria.

01 / ESCOPO

O que a equipe precisa produzir

Segundo o resumo fornecido pelo grupo, o trabalho reúne um programa em C, uma análise experimental e uma análise teórica. Cada algoritmo precisa aparecer nas três frentes.

Frente	Resultado esperado
Programa	Comandos 1: inversão; 2: busca sequencial; 3: binária iterativa; 4: binária recursiva; 5: encerrar.
Experimento	Cinco tamanhos: 1.000, 50.000, 100.000, 500.000 e 1.000.000. Pelo menos 100 execuções por tamanho; tempo médio; comparações e atribuições no pior caso.
Teoria	Função de custo $T(n)$, melhor, médio e pior caso; termo dominante; classificação em Grande O.
Comunicação	Tabelas, gráficos, metodologia reproduzível e discussão dos resultados.

Requisito relatado × melhoria sugerida

Relatado pelo grupo: os cinco tamanhos, o mínimo de 100 execuções, a análise e as entregas acima.

Sugerido neste guia: aquecimento, 10 rodadas por configuração, cronômetro de alta resolução, calibração de repetições, desvio-padrão e preservação dos tempos de cada rodada.

Essas melhorias tornam as conclusões mais defensáveis, mas não garantem uma nota específica. O critério de avaliação é o do professor.

Entrega informada na conversa

Código principal pelo RunCodes; relatório e códigos auxiliares pelo e-disciplinas. O resumo menciona 18/09: confirmem ano, horário e arquivos exigidos no enunciado ou na página da disciplina.

02 / PONTO DE PARTIDA

O projeto que existe hoje

Arquivo / evidência	Situação observada em 17/09/2026
src/algoritmos.c	Inversão implementada; binária recursiva implementada. Sequencial e binária iterativa ainda retornam 0 sem fazer busca.
src/main.c	Lê comandos e imprime SIM/NAO. Usa cópia para inverter, preservando o vetor ordenado das buscas.
experiments/auxiliar_inversao.c	Mede lote com clock(), usa contadores globais e grava CSV. Ainda não mede variação entre rodadas.
data/resultados_inversao.csv	Possui cinco linhas de resultados. As contagens coincidem com as fórmulas da inversão; faltam metadados para reproduzir os tempos.

Contrato que este guia adota

As buscas retornam **1 se encontrarem o valor e 0 se não encontrarem**, como o main atual espera. Não copiem exemplos que retornam índice e -1 sem alterar também quem chama a função: o índice zero seria interpretado como ausência, e -1 como presença.

A busca recursiva recebe limites inclusivos: **inicio = 0** e **fim = n - 1**. As duas buscas binárias exigem vetor em ordem crescente para a lógica apresentada aqui.

Nome do projeto

A captura mostra SCC0502-Trabalho-1; a pasta, o README e o remoto local usam SCC0201-Trabalho-1. O guia usa um título neutro. Confirmem o código correto da disciplina antes de fechar a capa do relatório.

03 / ORGANIZAÇÃO

Cada pasta tem uma função

Pasta	O que guardar
src/	main.c, algoritmos.c e algoritmos.h: implementação oficial, sem contadores.
experiments/	Um auxiliar por algoritmo; versões instrumentadas; rotinas de tempo compartilhadas, se adotadas.
data/	CSV bruto por rodada, resumos e metadados da execução.
analysis/	Derivações de $T(n)$, convenção de contagem e scripts de análise.
graphs/	Gráficos finais com títulos, legendas e unidades.
report/	Texto do relatório e PDF final.
submission/	Somente a versão preparada para envio conforme o enunciado.

Fluxo: função correta → experimento → CSV → análise e gráficos → relatório → entrega.

Divisão entre quatro integrantes

Pessoa 1: inversão e integração do main.

Pessoa 2: sequencial e padrão de tempo.

Pessoa 3: binária iterativa e padrão de contagem.

Pessoa 4: binária recursiva e organização dos gráficos/relatório.

Cada pessoa entrega código, testes, contagens, dados, teoria e texto do próprio algoritmo. A organização do relatório não transfere a escrita de todos para uma única pessoa. Combinem quem edita algoritmos.c a cada momento, pois ele é compartilhado.

04 / ANTES DE MEDIR

Primeiro, prove que funciona

Um programa incorreto pode parecer muito rápido. As duas buscas ainda incompletas do projeto sempre retornam 0; medir esse retorno não mede uma busca. Só iniciem a coleta depois dos testes abaixo.

Teste	Entrada / resultado esperado
Inversão par	[0,1,2,3] → [3,2,1,0].
Inversão ímpar	[0,1,2,3,4] → [4,3,2,1,0]. O centro fica no lugar.
Uma posição	[7] permanece [7]; busca de 7 retorna 1, de 8 retorna 0.
Buscas	Em [0,1,2,3,4], procurar 0, 2 e 4 retorna 1; procurar -1 e 5 retorna 0.
Repetidos	Em [1,1,2,2], buscar 1 ou 2 retorna 1. Não exigimos um índice específico.
Integração	Executar inversão e depois uma busca binária: a busca deve continuar correta.
Instrumentação	Versão original e contada dão o mesmo resultado sobre entradas equivalentes.

Como executar e conferir

Compile com avisos habilitados. Para cada caso, escreva a entrada e a saída esperada antes de executar. Se houver erro, corrija a lógica e repita os casos relacionados. Depois valide também as contagens à mão em $n = 1, 2, 3, 4$ e 5 .

```
gcc -std=c11 -Wall -Wextra -O2 src/main.c src/algoritmos.c -o programa.exe
```

No PowerShell: execute `.\programa.exe` e forneça os dados no formato pedido. Não adicione mensagens de menu ou instruções à saída se o juiz espera apenas os resultados.

05 / MECANISMOS

O que cada algoritmo faz

Inversão: trocar as extremidades

Dois índices começam nas pontas. A cada passo, uma variável temporária permite trocar os valores sem perder nenhum deles. O índice esquerdo avança e o direito recua. Em $n = 5$, são feitas as trocas (0,4) e (1,3); a posição 2 permanece no centro.

```
0 1 2 3 4  ->  4 1 2 3 0  ->  4 3 2 1 0
```

Sequencial: olhar uma posição por vez

Começa na posição zero e compara cada elemento com o alvo. Para ao encontrar ou ao terminar o vetor. Se o alvo é n e o vetor vai de 0 a $n-1$, todos os elementos são examinados.

Binária: descartar metade do intervalo

Com o vetor ordenado, examine o meio. Se o alvo for maior, descarte a metade esquerda; se for menor, descarte a direita. Exemplo: em $[0,1,2,3,4,5,6]$, procurar 7 visita os valores 3, 5 e 6; o intervalo então fica vazio.

Iteração e recursão: duas formas de repetir

A iterativa atualiza início e fim em um laço. A recursiva chama a si mesma com um intervalo menor. O caso-base **início > fim** encerra a busca ausente. O cálculo **início + (fim-início)/2** evita somar diretamente os limites.

Por que não buscar no vetor invertido?

A decisão de descartar metade supõe a ordem crescente. Recrie o vetor antes de cada experimento de busca; no programa principal, mantenha a estratégia atual de inverter uma cópia.

06 / DESENHO EXPERIMENTAL

Uma configuração por vez

Uma configuração é a combinação de **algoritmo + tamanho n + cenário**. Para a coleta principal, use os cinco tamanhos e um cenário de pior caso por algoritmo: são 20 configurações.

Algoritmo	Preparação	O que acontece
Inversão	$v[i] = i$	Sempre faz $\text{ piso}(n/2)$ trocas. Os valores não alteram a contagem.
Sequencial	$v[i] = i$; $\text{alvo} = n$	O alvo não existe; percorre o vetor inteiro.
Binária iterativa	$v[i] = i$; $\text{alvo} = n$	Segue a direita até esvaziar o intervalo.
Binária recursiva	$v[i] = i$; $\text{alvo} = n$	Mesmo caminho, com chamada final ao intervalo vazio.

Para os limites inclusivos e o meio arredondado para baixo usados neste guia, $\text{alvo} = n$ produz um caminho de comprimento máximo: **$k = \text{piso}(\log_2 n) + 1$** , para $n \geq 1$. Isso precisa ser revisto se a implementação mudar.

Prepare fora do cronômetro

```
int *v = malloc((size_t)n * sizeof *v);
if (v == NULL) { /* informar erro e encerrar */ }
for (int i = 0; i < n; i++) v[i] = i;
int alvo = n;
```

Um milhão de inteiros ocupa cerca de 4 MB quando $\text{sizeof}(\text{int}) = 4$. Use alocação dinâmica e verifique falhas. Ao terminar uma configuração, libere o vetor com `free(v)`.

Melhor caso e caso médio podem ser experimentos adicionais. Não misture cenários no mesmo resumo. Uma busca por um valor fixo do meio não representa, sozinha, o caso médio.

07 / INSTRUMENTAÇÃO

A regra de contagem do grupo

Definam a convenção antes de derivar fórmulas. Neste guia, contamos operações no código-fonte; não instruções de máquina nem incrementos dos próprios contadores.

Conta como comparação	Conta como atribuição
Cada avaliação de condição original: $i < n$, $\text{inicio} \leq \text{fim}$, $v[i] == p$, $v[\text{meio}] < p$.	Inicialização com valor, cópia e atualização: $i = 0$, $\text{meio} = \dots$, $v[i] = \dots$, $i++$, $j--$.
Inclui a condição final falsa do laço, quando ela ocorre.	$i++$ conta uma atribuição; a soma não vira outra atribuição.

Não entram: declaração sem inicialização, return, passagem de parâmetros, chamadas, aritmética isolada, preparação do vetor, malloc, impressão, estatísticas e contadores auxiliares. Se o professor pedir outro modelo, ajustem todas as fórmulas e os quatro contadores juntos.

O contador descreve o algoritmo original

Uma instrumentação pode usar while (1) e break para tornar cada teste explícito. Esses mecanismos adicionais não contam: cada incremento representa uma operação existente no algoritmo original. Zere os contadores antes de cada chamada completa.

```
typedef struct {
    long long comparacoes;
    long long atribuicoes;
} Contagem;

Contagem op = {0, 0}; /* no programa auxiliar */
```

Contagem não é tempo

Execute a versão contada uma vez por entrada determinística. Meça tempo com a função original. Cronometrar os contadores mede também o custo da instrumentação.

08 / INVERSÃO

Da troca à função de custo

Seja $m = \text{piso}(n/2)$. Existem duas inicializações, m avaliações verdadeiras do laço, uma avaliação falsa, três atribuições de troca e duas atualizações por iteração.

```
void inverte_conta(int *v, int n, Contagem *op) {
    int i = 0, j = n - 1, temp;
    op->atribuicoes += 2;
    for (;;) {
        op->comparacoes++;
        if (!(i < n / 2)) break;
        temp = v[i];
        v[i] = v[j];
        v[j] = temp;
        op->atribuicoes += 3;
        i++; j--;
        op->atribuicoes += 2;
    }
}
```

Medida	Fórmula	Para $n = 5$
Comparações $C(n)$	$m + 1$	3
Atribuições $A(n)$	$5m + 2$	12
Custo $T(n) = C(n) + A(n)$	$6m + 3$	15

Para os cinco tamanhos pares, $C(n) = n/2 + 1$ e $A(n) = 5n/2 + 2$. Para tamanhos ímpares, preserve o piso: não use uma contagem fracionária.

Melhor, médio e pior caso têm a mesma contagem para n fixo. O termo dominante é linear; portanto, $T(n)$ é $\Theta(n)$, e também $O(n)$. O espaço auxiliar da função é $O(1)$. A cópia feita pelo main usa $O(n)$, mas fica fora da função medida.

09 / SEQUENCIAL

Conte também a saída do laço

O exemplo retorna 1 ou 0, como o projeto. A instrumentação abaixo corresponde a um for que avança até achar o alvo ou terminar o vetor.

```
int sequencial_conta(int *v, int n, int p,
                    Contagem *op) {
    int i = 0;
    op->atribuicoes++;
    for (;;) {
        op->comparacoes++;
        if (!(i < n)) break;
        op->comparacoes++;
        if (v[i] == p) return 1;
        i++;
        op->atribuicoes++;
    }
    return 0;
}
```

Situação	C	A	T = C + A
Alvo ausente	$2n + 1$	$n + 1$	$3n + 2$
Alvo no índice j	$2(j + 1)$	$j + 1$	$3(j + 1)$
Melhor: índice 0	2	1	3

Com alvo ausente, o teste $i < n$ ocorre $n + 1$ vezes; a igualdade, n vezes. Encontrar o último elemento evita o teste final falso, então a contagem exata difere da ausência, embora ambos sejam $O(n)$.

Para sucesso igualmente provável em cada uma das n posições, a média de elementos visitados é $(n+1)/2$: $E[T] = 3(n+1)/2$. Esse caso médio pressupõe uma distribuição; se houver buscas ausentes, declare também sua probabilidade.

10 / BINÁRIA ITERATIVA

O intervalo diminui a cada passo

```
int binaria_conta(int *v, int n, int p,
                 Contagem *op) {
    int inicio = 0, fim = n - 1;
    op->atribuicoes += 2;
    for (;;) {
        op->comparacoes++;
        if (!(inicio <= fim)) break;
        int meio = inicio + (fim - inicio) / 2;
        op->atribuicoes++;
        op->comparacoes++;
        if (v[meio] == p) return 1;
        op->comparacoes++;
        if (v[meio] < p) inicio = meio + 1;
        else fim = meio - 1;
        op->atribuicoes++;
    }
    return 0;
}
```

No cenário $v[i] = i$ e $p = n$, cada uma das k iterações realiza três comparações e duas atribuições. Somam-se as duas inicializações e a comparação final do laço.

Cenário	C	A	T
Ausência: $k = \text{piso}(\log_2 n) + 1$	$3k + 1$	$2k + 2$	$5k + 3$
Encontra no primeiro meio	2	3	5
Sucesso após k iterações	$3k - 1$	$2k + 1$	$5k$

Em $n = 1.000$, $k = 10$: $C = 31$ e $A = 22$. Em $n = 1.000.000$, $k = 20$: $C = 61$ e $A = 42$. O vetor fica mil vezes maior, mas o caminho apenas dobra de comprimento.

11 / BINÁRIA RECURSIVA

Cada chamada tem seu próprio intervalo

```
int recursiva_conta(int *v, int inicio, int fim,
                  int p, Contagem *op) {
    op->comparacoes++;
    if (inicio > fim) return 0;
    int meio = inicio + (fim - inicio) / 2;
    op->atribuicoes++;
    op->comparacoes++;
    if (v[meio] == p) return 1;
    op->comparacoes++;
    if (v[meio] < p)
        return recursiva_conta(v, meio + 1, fim, p, op);
    return recursiva_conta(v, inicio, meio - 1, p, op);
}
```

Passe o mesmo ponteiro op pelas chamadas. Ele acumula o total; não zere os contadores dentro da função, pois isso apagaria as operações anteriores.

Cenário	C	A	T
Ausência: k chamadas não vazias	$3k + 1$	k	$4k + 1$
Encontra no primeiro meio	2	1	3
Sucesso após k chamadas	$3k - 1$	k	$4k - 1$

A ausência acrescenta uma chamada final com intervalo vazio. Ela faz uma comparação e nenhuma atribuição. Parâmetros de entrada não são contados como atribuições neste modelo.

Menos atribuições não significa menor tempo

A contagem escolhida não representa o custo de chamadas nem de controle da pilha. A recursiva pode ter tempos diferentes da iterativa, apesar da mesma ordem $O(\log n)$. O espaço auxiliar é $O(\log n)$ sem eliminação de chamadas de cauda pelo compilador; a iterativa usa $O(1)$.

12 / CRONOMETRAGEM

Meça um intervalo confiável

Uma busca binária isolada pode ser rápida demais para a resolução prática do relógio. Cronometre um **lote de R chamadas** e divida o tempo total por R. Assim, o custo de consultar o relógio é diluído.

No Windows

Use `QueryPerformanceCounter` para ler um contador de alta resolução e `QueryPerformanceFrequency` para conhecer quantos ticks representam um segundo. A duração é a diferença entre leituras dividida pela frequência. O exemplo completo está nas páginas 26–29. [1]

```
tempo_total_s = (fim.QuadPart - inicio.QuadPart)
                / (double)frequencia.QuadPart;
tempo_por_chamada_s = tempo_total_s / repeticoes;
```

O que fica dentro e fora

Fora do cronômetro	Dentro do cronômetro
Alocação, geração, cópia de preparação, aquecimento e calibração.	Laço de repetições e chamadas da função original.
Contagem de operações, validação, média, desvio-padrão, printf e CSV.	Nas buscas, um pequeno consumo do retorno pode ficar no lote para tornar o resultado observável.

O lote inclui algum custo do laço e das chamadas: o resultado é uma estimativa do custo por execução sob esse protocolo. Em algoritmos muito rápidos, esse custo pode importar; documente-o e mantenha a mesma estratégia nas buscas.

E o clock() atual?

Não é automaticamente inválido, mas sua resolução e semântica dependem da implementação. Não trate `CLOCKS_PER_SEC` como garantia de precisão real. Se trocar o relógio, refaça a série completa; não uma tempos obtidos com métodos diferentes. [1]

13 / REPETIÇÕES

R chamadas formam uma rodada

1. Aqueça

Execute, por exemplo, 20 chamadas sem medir. Isso reduz o peso do primeiro acesso a código e dados. O experimento passa a caracterizar execuções repetidas com dados reutilizados, não uma operação com cache frio.

2. Calibre R para cada algoritmo e tamanho

Comece com $R = 100$. Meça um lote piloto. Se durar menos que uma meta como 0,05 s, dobre R e tente novamente. Pare quando atingir a meta ou um limite de segurança. O piloto não entra nas estatísticas finais.

```
R = 100;
repetir:
  medir R chamadas;
  se tempo_total >= 0.05 s: terminar calibracao;
  se dobrar R exceder limite: registrar falha;
  R = 2 * R;
```

3. Fixe R e colete M rodadas

Uma sugestão é $M = 10$ rodadas, todas com o R calibrado para aquela configuração. Se $R = 10.000$, cada rodada resume 10.000 chamadas; dez rodadas representam 100.000 chamadas medidas. Aquecimento e pilotos são adicionais.

Por que R pode mudar?

O que será comparado é o tempo por chamada. A inversão de um vetor grande precisa de menos repetições para formar um lote longo; uma busca binária precisa de mais. A regra de calibração deve ser a mesma e o R utilizado deve ser salvo.

A fórmula atual $\max(100, 100000000/n)$ é uma heurística razoável para trabalho linear. Não a copie automaticamente para as buscas binárias: a duração delas não cresce proporcionalmente a n .

14 / EXECUÇÃO

A sequência de uma configuração

Passo	Ação	Por quê
1	Alocar e preencher $v[i] = i$.	Definir uma entrada reproduzível.
2	Testar o resultado original e instrumentado.	Impedir que um erro vire dado.
3	Preparar novamente o vetor.	Restaurar a entrada após os testes.
4	Aquecer e calibrar R.	Obter um lote mensurável.
5	Medir M lotes e guardar cada duração.	Quantificar a variação observada.
6	Dividir cada lote por R.	Obter o tempo por chamada.
7	Contar uma execução separada.	Registrar C e A sem contaminar o tempo.
8	Calcular resumo, salvar e liberar memória.	Permitir auditoria e repetição.

Repetir a inversão no mesmo vetor funciona?

Sim. Uma inversão o deixa decrescente, a próxima o restaura. A função percorre os mesmos índices em ambos os casos. Antes de cada rodada, você pode preencher novamente fora do cronômetro e aquecer com número par de inversões.

Evite medir código eliminado

Use as funções de `src/algoritmos.c` compiladas separadamente do auxiliar, sem LTO. Nas buscas, consuma os retornos em uma soma local e use-a depois do lote; se necessário, use um destino `volatile` no auxiliar. Não torne o vetor inteiro `volatile`, pois isso altera os acessos medidos. Verifique também o resultado da inversão após o lote.

Mantenha a mesma máquina e as mesmas opções de compilação. Evite executar os quatro experimentos simultaneamente e registre se houve mudança de carga ou energia.

15 / ESTATÍSTICA

Média e variação, sem mistério

Se a rodada r durou B_r segundos e contém R chamadas, o valor daquela rodada é $t_r = B_r/R$. Primeiro calcule os tempos por chamada; depois resuma as M rodadas.

```
media = (t1 + t2 + ... + tM) / M

s = sqrt( soma((tr - media)^2) / (M - 1) )

CV = 100 * s / media /* quando media > 0 */
```

O desvio-padrão amostral s descreve a dispersão dos **tempos médios por chamada de cada rodada**. Ele não mede a variação de cada chamada individual dentro do lote. Dez rodadas seguidas também não garantem independência estatística.

Um exemplo didático

Rodada	Lote de 10.000 chamadas	Tempo por chamada
1	0,100 s	10 μ s
2	0,110 s	11 μ s
3	0,090 s	9 μ s

A média é 10 μ s e o desvio-padrão é 1 μ s. Esses números são ilustrativos, não medições deste projeto. Para apresentar em microssegundos, multiplique segundos por 1.000.000.

Como interpretar a diferença

Se duas médias forem próximas em relação à variação, descreva o resultado como inconclusivo para aquela diferença. Não declare uma versão vencedora por uma única rodada. Média $\pm$ desvio-padrão não é automaticamente um intervalo de confiança.

16 / DADOS

CSV que permite conferir o trabalho

Guarde os dados de cada rodada e um resumo por configuração. O arquivo atual de inversão é um resumo antigo; preserve sua origem se gerar uma nova série com outro método.

Arquivo bruto: uma linha por rodada

```
algoritmo,n,cenario,rodada,repeticoes,tempo_total_s
inversao,1000,padrao,1,100000,0.053000000
```

A linha acima reaproveita valores de escala apenas para ilustrar o formato; não é uma nova rodada medida. Use ponto decimal no arquivo e vírgula para separar campos. Grave segundos com precisão suficiente, como %.12e.

Resumo: uma linha por configuração

```
algoritmo,n,cenario,rodadas,repeticoes,media_s,dp_s,
comparacoes,atribuicoes
```

O cabeçalho acima foi quebrado visualmente: no CSV real ele ocupa uma só linha. Com quatro algoritmos, cinco tamanhos e um cenário, são 20 linhas de resumo e, com dez rodadas, 200 linhas brutas.

Metadados: registre junto aos CSVs

Data, responsável, processador, memória, sistema operacional, versão do compilador, comando de compilação, temporizador, aquecimento, número de rodadas, critério de calibração, definição da entrada e identificação do commit testado.

```
gcc --version
git rev-parse HEAD
git status --short
```

O commit sozinho não descreve alterações ainda não commitadas. Prefira medir uma versão salva em commit; caso contrário, registre também essa condição. Contagens determinísticas se repetem entre máquinas; tempos não precisam coincidir.

17 / GRÁFICOS

Mostre crescimento, não só números

Gráfico	Eixos e leitura
Tempo por chamada	$x = n$; $y =$ média em segundos ou μs . Barras de erro = desvio-padrão entre rodadas, explicitado na legenda.
Comparações	$x = n$; $y = C(n)$ de uma chamada no pior caso. Sobreponha fórmula e contagem observada.
Atribuições	$x = n$; $y = A(n)$, com a mesma convenção para todos.
Binárias em detalhe	Separe iterativa e recursiva em um gráfico próprio para que a escala linear não esconda os valores.

Passo a passo em uma planilha

1. Importe o CSV com separador vírgula e ponto como decimal.
2. Confira se n e tempo são números, não texto.
3. Crie um gráfico de dispersão XY, usando n no eixo x .
4. Separe séries por algoritmo e cenário; acrescente unidades.
5. Use os valores da coluna `dp_s` nas barras de erro, se a ferramenta permitir.
6. Exporte a figura e salve em `graphs/` junto do arquivo ou script que a gera.

Os tamanhos não são igualmente espaçados: $1.000 \rightarrow 50.000$ difere de $500.000 \rightarrow 1.000.000$. Um gráfico de categorias igualmente espaçadas pode distorcer essa leitura. Se usar escala logarítmica, informe isso nos eixos.

O que procurar

Ao dobrar n , contagens lineares aproximadamente dobram. Em buscas binárias, o número máximo de passos aumenta em cerca de uma unidade. Os tempos podem mudar por cache e outros efeitos: a teoria prevê ordem de crescimento, não segundos exatos.

18 / TEORIA

Como escrever $T(n)$ com clareza

Neste guia, **$T(n) = C(n) + A(n)$** : cada operação contada tem peso unitário. Esse é um modelo declarado, não uma medida completa do processador. Se quiser usar pesos distintos, escreva $T(n) = aC(n) + bA(n)$ e explique a escolha.

Algoritmo	Melhor caso	Médio ¹	Pior caso
Inversão	$\Theta(n)$	$\Theta(n)$	$\Theta(n)$
Sequencial	$\Theta(1)$	$\Theta(n)$	$\Theta(n)$
Binária iterativa	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$
Binária recursiva	$\Theta(1)$	$\Theta(\log n)$	$\Theta(\log n)$

¹ Para as buscas, supondo sucesso igualmente provável entre n chaves distintas. A distribuição deve ser explicitada. A inversão independe dos valores. Θ descreve uma ordem justa de crescimento; O fornece um limite superior. As classificações em Θ da tabela também satisfazem o O correspondente.

Desenvolvendo a recorrência

No caminho máximo de ausência da binária recursiva, para $n \geq 1$, o intervalo da direita tem $\text{piso}(n/2)$ elementos. Sob nossa contagem: **$T(n) = T(\text{piso}(n/2)) + 4$** , com **$T(0) = 1$** . Após k níveis não vazios: $T(n) = 4k + 1$, onde $k = \text{piso}(\log_2 n) + 1$.

Um parágrafo que cada integrante deve escrever

Explique a entrada de pior caso; conte quantas vezes cada operação ocorre; some os termos; identifique o dominante; conclua a ordem assintótica. Depois compare a fórmula com os CSVs. Para sucesso em n posições uniformes, explique a hipótese ao discutir o caso médio.

Gráficos apoiam a análise, mas não provam a complexidade para todo n . A justificativa vem da lógica do algoritmo e de sua contagem ou recorrência.

19 / GITHUB

Primeiro acesso, uma vez por pessoa

Git guarda o histórico local. GitHub hospeda a cópia compartilhada. A branch **main** será a linha de trabalho da equipe, conforme a decisão do grupo; não é necessário abrir uma branch por tarefa neste fluxo.

1. Prepare o acesso

Instale Git e use um editor de sua preferência. Entre no GitHub e aceite o convite de colaborador, se necessário. Um repositório público permite leitura, mas a permissão de escrita precisa ser concedida. Autentique pelo fluxo oferecido pelo Git Credential Manager ou GitHub Desktop.

2. Configure a autoria no seu computador

```
git config --global user.name "Seu Nome"
git config --global user.email "seu-email-ou-noreply"
```

Substitua os valores. O e-mail pode ser o endereço noreply disponibilizado na sua conta do GitHub. Essa configuração identifica o autor; não realiza login.

3. Clone o repositório existente

```
git clone https://github.com/mathbmach/SCC0201-Trabalho-1.git
cd SCC0201-Trabalho-1
git status
git branch --show-current
```

O endereço é o remoto observado na cópia local. Se o grupo tiver renomeado o repositório mostrado na captura, copie o endereço atual em Code → HTTPS. Clone somente uma vez; quem já tem a pasta usa pull para atualizá-la. Não execute git init dentro do projeto clonado. [2]

Alternativa visual

No GitHub Desktop, use File → Clone repository, escolha o projeto ou sua URL e a pasta local. Verifique Current Branch: main. Os conceitos de commit, pull e push são os mesmos. [3]

A rotina diária na main

1. Antes de editar: atualize

```
git status
git switch main
git pull --ff-only origin main
```

Comece com a árvore de trabalho limpa. Se houver alterações suas, finalize e salve-as em commit antes de integrar mudanças remotas. Se --ff-only recusar porque os históricos divergiram, siga a próxima página. Combine no grupo qual função e quais arquivos você vai editar. [4]

2. Edite, teste e confira o que mudou

```
git diff
git status
git add experiments/auxiliar_inversao.c
git diff --staged
git commit -m "Adiciona rodadas ao experimento de inversao"
```

O caminho é um exemplo: adicione apenas os arquivos da sua alteração. add seleciona o conteúdo; commit salva um marco local. Prefira mensagens que expliquem o resultado. Uma alteração no código deve vir acompanhada da validação correspondente.

3. Integre o que chegou e publique

```
git pull --rebase origin main
git push origin main
git status
```

Use rebase aqui para reaplicar seus commits ainda locais sobre as mudanças recebidas. Depois de integrar, compile e teste novamente antes do push. Se alguém publicar enquanto você trabalha, repita pull, resolva o que for necessário e então envie. [4,5]

Confirme no GitHub que o commit e os arquivos apareceram. Avise à equipe o que está pronto. Commit sem push continua apenas na sua máquina.

Conflitos têm solução

Um conflito significa que o Git não conseguiu juntar automaticamente duas alterações. Ele é comum quando duas pessoas editam as mesmas linhas de algoritmos.c.

Durante pull --rebase

1. Use git status para identificar o arquivo.
2. Abra o arquivo e leia os dois trechos entre os marcadores.
3. Converse com quem alterou a função e produza uma versão correta.
4. Remova todos os marcadores; compile e teste.
5. Marque a resolução e continue. Repita se houver mais conflitos. [5]

```
git add src/algoritmos.c
git rebase --continue
# Se precisar cancelar o rebase em andamento:
git rebase --abort
```

No rebase, os rótulos de “nosso” e “deles” podem confundir. Leia o conteúdo, não escolha um lado automaticamente. --abort volta ao estado anterior ao rebase; não publica nada. Não use push --force para contornar rejeições na main. [5]

Enviei algo errado

Para uma correção simples, edite e faça outro commit. Para desfazer um commit já publicado, use revert: ele cria um novo commit inverso e preserva o histórico. Confira o identificador com git log --oneline; substitua HASH abaixo pelo commit correto. [6]

```
git revert HASH
# Teste a versao resultante antes de enviar.
git push origin main
```

Dados e arquivos compartilhados

Escolham uma pessoa para a coleta final. Evitem sobrescrever o mesmo CSV com medições de máquinas diferentes. Versionem fontes, scripts, dados e relatório; executáveis e arquivos temporários devem ficar fora. Se o push for negado por permissão, confirmam conta e convite, sem recriar o repositório.

22 / RELATÓRIO

Transforme resultados em argumentos

Parte	O que escrever
Introdução	Objetivo, algoritmos estudados e perguntas da comparação.
Metodologia	Entradas, cenários, cinco tamanhos, relógio, R, M, aquecimento, ambiente, flags e convenção de contagem.
Cada algoritmo	Como funciona; código relevante; $T(n)$; melhor/médio/pior; tabela e discussão do experimento.
Comparação	Crescimento linear versus logarítmico; iterativa versus recursiva; diferenças entre contagem e tempo.
Conclusão	O que os dados sustentam, limitações e síntese da relação entre teoria e prática.
Reprodução	Comandos, nomes dos arquivos e commit usado para gerar a coleta final.

Como discutir sem exagerar

Em vez de “a binária é sempre melhor”, escreva em quais tamanhos e condições ela teve menor tempo. Mencione que o vetor já estava ordenado e que o custo de ordenação ficou fora da medição.

Se o tempo não crescer como esperado, confira resolução, retorno não utilizado, otimização, máquina, carga e erros de lógica. Não apague medições inconvenientes sem uma regra e justificativa registradas.

Padrão de contribuição individual

Cada membro entrega um texto que explique seu algoritmo, uma derivação de custo, dados rastreáveis e pelo menos uma interpretação. Outra pessoa revisa. Quem organiza o PDF uniformiza estilo e referências, sem precisar reconstruir os experimentos dos colegas.

23 / FECHAMENTO

Checklist para a equipe

Código e integração

- ☐ As quatro funções estão completas e retornam o contrato esperado.
- ☐ As buscas funcionam antes e depois do comando de inversão.
- ☐ Cabeçalho, nomes e parâmetros coincidem entre os arquivos.
- ☐ A saída segue exatamente o enunciado e o programa compila sem avisos relevantes.
- ☐ Alocações, leitura de entrada e limites foram revisados.

Experimentos e análise

- ☐ Os cinco tamanhos e as repetições mínimas foram contemplados.
- ☐ Tempo usa função original; contagem usa instrumentação separada.
- ☐ As fórmulas conferem com casos pequenos e com os CSVs.
- ☐ Rodadas, média, variação e unidades estão identificadas.
- ☐ Metadados, código medido e dados brutos estão preservados.
- ☐ Os gráficos representam os números reais, sem exemplos didáticos misturados.

Entrega

- ☐ Cada membro revisou sua seção e outra seção do relatório.
- ☐ A main está atualizada com a versão final testada.
- ☐ A pasta submission contém exatamente o conjunto solicitado.
- ☐ A versão preparada para envio foi compilada e testada novamente.
- ☐ Nome da disciplina, integrantes, números USP, prazo e formato foram conferidos.
- ☐ O responsável por cada envio confirmou o recebimento na plataforma.

Próximos passos no estado atual

Concluir sequencial e binária iterativa; validar o conjunto; evoluir o auxiliar de inversão; replicar o método nos outros três; fazer a coleta final em uma máquina; consolidar gráficos e relatório. O guia não altera essas implementações automaticamente.

Um experimento completo em C

Exemplo proposto para Windows: mede a inversão original e grava dez rodadas por tamanho. Salve a sequência das quatro páginas em **experiments/exemplo_inversao.c**. Inclua as partes na ordem, sem repetir cabeçalhos de página. O código usa o `inverte` já existente.

```
#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <limits.h>
#include "../src/algoritmos.h"

typedef struct { long long c, a; } Contagem;

static void preencher(int *v, int n) {
    for (int i = 0; i < n; ++i) v[i] = i;
}

static Contagem contar(int *v, int n) {
    Contagem op = {0, 2};
    int i = 0, j = n - 1, temp;
    for (;;) {
        op.c++;
        if (!(i < n / 2)) break;
        temp = v[i]; v[i] = v[j]; v[j] = temp;
        i++; j--;
        op.a += 5;
    }
    return op;
}

static int validar(int *v, int n, int invertido) {
    for (int i = 0; i < n; ++i)
        if (v[i] != (invertido ? n - 1 - i : i))
            return 0;
    return 1;
}
```

APÊNDICE / 2 DE 4

Relógio e medição do lote

A frequência é consultada uma vez no main. A função `lote` mede apenas o laço de chamadas e verifica falhas do relógio. Para cada rodada, preparar repõe a entrada e faz 20 inversões de aquecimento, um número par.

```
static double lote(int *v, int n, int r,
                  LARGE_INTEGER freq) {
    LARGE_INTEGER inicio, fim;
    if (!QueryPerformanceCounter(&inicio)) exit(2);
    for (int i = 0; i < r; ++i) inverte(v, n);
    if (!QueryPerformanceCounter(&fim)) exit(2);
    return (double)(fim.QuadPart - inicio.QuadPart)
           / (double)freq.QuadPart;
}

static void preparar(int *v, int n) {
    preencher(v, n);
    for (int i = 0; i < 20; ++i) inverte(v, n);
}

int main(void) {
    const int ns[] = {1000, 50000, 100000, 500000,
                      1000000};
    const int M = 10;
    LARGE_INTEGER freq;
    if (!QueryPerformanceFrequency(&freq) ||
        freq.QuadPart <= 0) return 2;
    FILE *bruto = fopen("data/exemplo_rodadas.csv", "w");
    FILE *res = fopen("data/exemplo_resumo.csv", "w");
    if (!bruto || !res) {
        if (bruto) fclose(bruto);
        if (res) fclose(res);
        return 3;
    }
    fprintf(bruto, "algoritmo,n,rodada,repeticoes,"
               "tempo_total_s\n");
    fprintf(res, "algoritmo,n,rodadas,repeticoes,media_s,"
              "dp_s,comparacoes,atribuicoes\n");
```

APÊNDICE / 3 DE 4

Validar, calibrar e coletar

O piloto dobra R até 0,05 s e não entra nos CSVs finais. A validação de paridade confirma se um número ímpar de inversões deixou o vetor invertido. A contagem é feita separadamente.

```
for (int t = 0; t < 5; ++t) {
    int n = ns[t], R = 100;
    int *v = malloc((size_t)n * sizeof *v);
    if (!v) return 4;
    preencher(v, n);
    inverta(v, n);
    if (!validar(v, n, 1)) return 5;
    preencher(v, n);
    Contagem op = contar(v, n);
    long long m = n / 2;
    if (!validar(v, n, 1) || op.c != m + 1 ||
        op.a != 5 * m + 2) return 6;
    for (;;) {
        preparar(v, n);
        double b = lote(v, n, R, freq);
        if (!validar(v, n, R % 2)) return 7;
        if (b >= 0.05) break;
        if (R > INT_MAX / 2) return 8;
        R *= 2;
    }
    double tempos[10], media = 0.0;
    for (int r = 0; r < M; ++r) {
        preparar(v, n);
        double b = lote(v, n, R, freq);
        if (!validar(v, n, R % 2)) return 7;
        tempos[r] = b / R;
        media += tempos[r];
        fprintf(bruto, "inversao,%d,%d,%d,%.12e\n",
            n, r + 1, R, b);
    }
    media /= M;
}
```

Os retornos 2–10 representam falhas; não use uma coleta interrompida como série completa. Para uma versão final do auxiliar, acrescente mensagens de diagnóstico e centralize a liberação de recursos nos caminhos de erro.

APÊNDICE / 4 DE 4

Salvar e adaptar o exemplo

```

double soma = 0.0;
for (int r = 0; r < M; ++r) {
    double d = tempos[r] - media;
    soma += d * d;
}
double dp = sqrt(soma / (M - 1));
fprintf(res, "inversao,%d,%d,%d,%.12e,%.12e,"
          "%lld,%lld\n",
          n, M, R, media, dp, op.c, op.a);
free(v);
}
int erro = ferror(bruto) || ferror(res);
if (fclose(bruto) != 0) erro = 1;
if (fclose(res) != 0) erro = 1;
return erro ? 10 : 0;
}

```

Na raiz do repositório, com GCC disponível e data/ existente, compile em uma única linha. Os dois CSVs exemplo serão substituídos a cada execução: copie ou renomeie coletas que quiser preservar.

```

gcc -std=c11 -O2 -Wall -Wextra
  experiments/exemplo_inversao.c src/algoritmos.c
  -o exemplo_inversao.exe -lm

.\exemplo_inversao.exe

```

Una as três primeiras linhas no terminal. Não inclua src/main.c: o auxiliar já possui seu próprio main. Não use -flto nesta proposta.

Adaptação para as buscas

Mantenha a sequência, os tamanhos, o relógio e as estatísticas. Troque `inverte` pela busca original, use `p = n`, valide retorno 0, substitua `contar` pela instrumentação correspondente e mude os nomes dos CSVs. A recursiva é chamada com `(v, 0, n-1, p)`. Consuma os retornos, como explicado na página 16.

REFERÊNCIAS

Bases e decisões a confirmar

Material do grupo. Conversa “Divisão detalhada das contribuições”; captura do repositório; README, src/main.c, src/algoritmos.c, src/algoritmos.h, auxiliar_inversao.c e CSV local, consultados em 17/09/2026. As fórmulas foram derivadas das implementações descritas neste guia.

- [1] [Microsoft Learn · Temporização de alta resolução](#)
- [2] [GitHub Docs · Receber alterações de um repositório](#)
- [3] [GitHub Docs · Clonar pelo GitHub Desktop](#)
- [4] [Git · pull](#)
- [5] [Git · rebase](#)
- [6] [Git · revert](#)

Documentação técnica consultada em 17/09/2026. Os títulos acima são links clicáveis. As recomendações de organização e os exemplos didáticos foram preparados para este projeto.

Confirmem antes da entrega

Enunciado: entrada, saída, limites de n , formato do envio e prazo.

Identificação: SCC0201 ou SCC0502; nomes e números USP.

Método: convenção de contagem e distribuição para o caso médio.

Coleta: máquina, compilador, commit, responsável e destino dos dados.

O objetivo final

Qualquer integrante deve conseguir explicar o que foi contado, o que foi cronometrado, por que a entrada força o pior caso e como outra pessoa reproduz o resultado. Esse entendimento é a base de um trabalho consistente.